

C64uRemote für M5StickC Plus2

Technische Dokumentation

Version 1.0

Inhaltsverzeichnis

Überblick	2
Hardware	2
Zielformat	2
Zwei getrennte I ² C-Busse	2
Erkennung beim Start	3
Programmierungsumgebung	3
PlatformIO	3
Board-Einstellungen – die Falle mit dem dunklen Display	3
PSRAM	4
Speicherhaushalt	4
Konfiguration (build_env.h)	4
WLAN-Subsystem	4
Laufzeitkonfiguration statt Compile-Zeit	4
NVS-Layout	5
Kartenformat	5
Setup-Portal	5
NFC-Subsystem	6
Abfragestrategie	6
Kartentypen	6
Datenformat	6
Kommandokarten	7
Was der Stick nicht kann	7
Softwarearchitektur	7
Zustandsmodell	7
Navigation	7
Rendering	8
ReST-Anbindung an den C64 Ultimate	8
Bedienlogik	8
Tasten	8
Blockierende Aufrufe	9
PowerOff-Absicherung	9
Persistenz	9
Projektstruktur	9
Fehlerdiagnose	9
Quellen	10
Lizenz	10

Überblick

Diese Fassung von C64uRemote läuft auf einem **M5StickC Plus2** und steuert einen **Commodore 64 Ultimate** über dessen ReST-API. Gegenüber der ersten StickC-Version sind drei Bausteine dazugekommen, alle aus der M5Stack-Core- Fassung portiert:

- **NFC über ein Unit RFID2** am Grove-Port – WLAN-Karten und Befehlskarten lesen, beide auch schreiben
- **WLAN-Verwaltung zur Laufzeit** – bis zu vier Netze im NVS statt fester Werte aus `build_env.h`
- **Setup-Portal** – eigener Accesspoint mit kleiner Weboberfläche

Nicht portiert wurde alles, was eine microSD-Karte braucht: Dateibrowser, Upload an den C64, Dump und Restore von Karten. Der Stick hat keinen Kartenschacht.

Das Kartenformat ist bewusst identisch zu M5Dial und M5Stack Core. Eine Karte, die an einem der Geräte entsteht, funktioniert an allen.

Hardware

Ziellplattform

SoC	ESP32-PICO-V3-02, 240 MHz, Dual Core
Flash	8 MB (im Gehäuse)
PSRAM	2 MB (im Gehäuse)
Display	ST7789, 135 × 240, im Betrieb quer genutzt (240 × 135)
Backlight	GPIO 27, per PWM
Stromhaltepin	GPIO 4 – muss HIGH sein, sonst schaltet sich das Gerät im Akkubetrieb ab
Bedienung	Taste A (GPIO 37), Taste B (GPIO 39), Power-Taste über den PMU

Der ESP32-PICO-V3-02 ist wichtig zu kennen: Er unterscheidet sich vom PICO-D4 des alten M5StickC durch das **integrierte PSRAM** und ein anderes Gehäusekennzeichen im eFuse (`pkg_ver == 6`). Genau daran erkennt M5GFX den Plus2 – siehe *Programmierung*.

Zwei getrennte I²C-Busse

Das ist die zentrale Verdrahtungsentscheidung dieser Fassung:

Bus	Pins	Teilnehmer	Adresse
Wire	SDA = G0, SCL = G26	Hat MiniJoyC (HAT-Port)	0x54
Wire1	SDA = G32, SCL = G33	Unit RFID2 / WS1850S (Grove-Port)	0x28

Beide Geräte laufen dadurch parallel, ohne sich in die Quere zu kommen. Der MFRC522-Treiber bekommt seine Businstanz im Konstruktor mit:

```
MFRC522_I2C rfid(kRfidAddr, -1, &Wire1);
```

`Wire1.begin(32, 33, 100000UL)` muss vor `initRfid()` laufen – `PCD_Init()` startet den Bus **nicht** selbst.

Der MiniJoyC wird ohne Bibliothek angesprochen: `miniJoyBegin()` startet den Bus und pingt die Adresse an, `miniJoySetLedColor()` schreibt drei Bytes auf Register 0x40, gelesen wird über `joyReadRegister()`. Das spart eine Abhängigkeit und die Mehrdeutigkeits-Warnungen aus deren Quelltext.

Erkennung beim Start

```
Wire.begin(0, 26, 100000)    -> MiniJoyC anpingen  -> miniJoyReady  
Wire1.begin(32, 33, 100000) -> initRfid()        -> app.rfidReady
```

initRfid() prüft zuerst mit einer leeren I²C-Transaktion, ob unter 0x28 überhaupt etwas antwortet, und ruft erst dann PCD_Init(). Das Versionsregister wird nur noch protokolliert und **nicht** ausgewertet: der WS1850S meldet dort nicht immer denselben Wert wie ein echter MFRC522.

Programmierungsumgebung

PlatformIO

```
cp src/build_env.h.example src/build_env.h # ausfüllen  
pio run -t upload  
pio device monitor
```

In VS Code genügt es, den Projektordner zu öffnen; die PlatformIO-IDE legt .vscode/c_cpp_properties.json selbst an. .vscode/extensions.json und .vscode/settings.json liegen im Projekt und schlagen beim Öffnen die PlatformIO-Erweiterung vor.

Board-Einstellungen - die Falle mit dem dunklen Display

Für den StickC Plus2 gibt es (Stand espressif32 6.9) **keinen eigenen Board-Eintrag**. Man nimmt deshalb den des M5StickC – aber nicht unverändert.

Das Board-Manifest m5stick-c.json setzt -DARDUINO_M5Stick_C. M5GFX benutzt diese Kennung in init_impl() als Vorgabe:

```
#elif defined ( ARDUINO_M5STICK_C ) || defined ( ARDUINO_M5Stick_C )  
    nvs_board = board_t::board_M5StickC;           // = 3
```

und reicht sie an autodetect() weiter. Dort steht für den PICO-V3-02:

```
else if (pkg_ver == 6)           // PICOV3_02 (StickCPlus2 / ATOM PSRAM)  
{  
    if (board == 0 || board == board_t::board_M5StickCPlus2)
```

Mit dem Wert 3 ist die Bedingung falsch. Der Zweig wird übersprungen, es wird **überhaupt kein Panel initialisiert** und der Stromhaltepin G4 bleibt ungesetzt. Ergebnis: der Bildschirm bleibt dunkel, ohne Fehlermeldung. Die Kennung ist also keine Vorauswahl, sondern eine Festlegung.

Die platformio.ini korrigiert das an drei Stellen:

```
board_build.extra_flags =                               ; -DARDUINO_M5Stick_C entfernen
```

```
build_flags =  
    -DM5GFX_BOARD=board_t::board_M5StickCPlus2  
    -DBOARD_HAS_PSRAM
```

```
build_unflags =  
    -DARDUINO_M5Stick_C
```

M5GFX_BOARD wird vor der ganzen #elif-Kette abgefragt und hat damit Vorrang. Zusätzlich setzt setup():

```
cfg.fallback_board = m5::board_t::board_M5StickCPlus2;
```

Zur Kontrolle: im seriellen Monitor muss [Autodetect] M5StickCPlus2 stehen.

M5GFX legt ein einmal erkanntes Board im NVS ab und liest es beim nächsten Start **vor** allen Compile-Zeit-Kennungen wieder ein. Steht dort ein falscher Wert, hilft nur pio run -t erase – dabei gehen auch die gespeicherten WLAN-Zugänge verloren.

PSRAM

Der PICO-V3-02 hat 2 MB PSRAM im Gehäuse, aber der Arduino-Core ruft `psramInit()` nur auf, wenn `BOARD_HAS_PSRAM` definiert ist. Ohne diese Kennung liefert `ps_malloc()` schlicht `NULL`:

```
void *ps_malloc(size_t size){
    if(!spiramDetected){ return NULL; }
    ...
}
```

Deshalb steht `-DBOARD_HAS_PSRAM` in den `build_flags`. Der Code verlässt sich trotzdem nicht darauf:

```
uint16_t* allocFrameBuffer(size_t bytes) {
    uint16_t* buffer = nullptr;
    if (psramFound()) buffer = static_cast<uint16_t*>(ps_malloc(bytes));
    if (buffer == nullptr) buffer = static_cast<uint16_t*>(malloc(bytes));
    return buffer;
}
```

Speicherhaushalt

Puffer	Größe	Lage
M5Canvas (Vollbild-Sprite)	240 × 135 × 2 = 64,8 kB	intern
plainLogoPixels	64,8 kB	PSRAM, sonst intern
boxedLogoPixels	64,8 kB	PSRAM, sonst intern

Solange das **Setup-Portal** läuft, werden die beiden Logo-Caches freigegeben: Accesspoint und Webserver brauchen den internen Heap. Drei Stellen gehören dabei zusammen und müssen gemeinsam betrachtet werden, wenn dort jemand etwas ändert:

- `startPortal()` ruft `releaseLogoCache()`
- `stopPortal()` ruft `ensureLogoCache()`
- `render()` hat einen Nullzeiger-Zweig für das Startbild ohne Cache

Schlägt die Belegung fehl, läuft das Gerät ohne Logo-Effekte weiter und meldet *WENIG SPEICHER*. Eine frühere Fassung blieb an dieser Stelle in einer Endlosschleife hängen – ein dunkler Bildschirm ohne jeden Hinweis.

Konfiguration (build_env.h)

```
#define C64U_WIFI_SSID      "MeinWLAN"
#define C64U_WIFI_PASSWORD "MeinWlanPasswort"
#define C64U_TARGET_HOST   "192.168.0.64"
#define C64U_TARGET_PASSWORD ""
```

Diese Werte sind **nur noch Startwerte**. `loadNetConfig()` liest zuerst den NVS; nur wenn dort noch nie etwas gespeichert wurde (Schlüssel `wcount` fehlt), wird der Eintrag aus `build_env.h` als erstes Profil übernommen. Die Felder dürfen deshalb auch leer bleiben.

WLAN-Subsystem

Laufzeitkonfiguration statt Compile-Zeit

```
struct WifiProfile { String ssid; String pass; };
WifiProfile gWifiProfiles[kWifiProfileMax]; // kWifiProfileMax = 4
size_t      gWifiCount = 0;
size_t      gWifiTry   = 0;
String      gTargetHost, gTargetPass;
```

wifiAddProfile() sortiert ein Netz nach vorn (zuletzt benutzt zuerst); ist die Liste voll, fällt der letzte Eintrag heraus. beginWiFi() nimmt gWifiProfiles[gWifiTry] und schaltet danach weiter, sodass serviceWiFi() alle 10 s ein anderes Netz probiert, bis eines antwortet.

Beim Start ruft setup() einmal wifiPickBestProfile(): ein blockierender Scan, der das bekannte Netz mit dem stärksten Signal nach vorn holt. Das lohnt sich nur bei mehr als einem gespeicherten Netz und wird sonst übersprungen.

NVS-Layout

Zwei Namensräume, damit ein *Factory Reset* der Anzeige-Einstellungen die Zugangsdaten nicht mitnimmt:

c64uremote – Einstellungen

Schlüssel	Inhalt
anim_on, fx_mode, anim_spd, fx_time, st_time, bright	Anzeige
joy_ovl, joy_led, joy_led_br, joy_thr	MiniJoyC
auto_nfc	Abstand der Hintergrundabfrage
card_cnf	Abfragezeit der PowerOff-Befehlskarte in Sekunde

c64unet – Netz

Schlüssel	Inhalt
host, hostpw	Adresse und Passwort des c64u
wcount	Anzahl gespeicherter Netze (0-4)
s0...s3	SSIDs
p0...p3	Passwörter

wcount dient zugleich als Merker „hier wurde schon einmal gespeichert“.

Kartenformat

WLAN-Karten benutzen das Schema aus dem WLAN-QR-Code:

WIFI:S:MeinNetz;T:WPA;P:geheim;;

parseWifiText() wertet die Felder S: und P: aus, T: wird ignoriert (der Typ ergibt sich aus dem Passwort). Sonderzeichen sind mit Backslash geschützt; der Parser arbeitet zeichenweise mit Escape-Zustand, sodass ; und : im Passwort funktionieren. wifiCardText() ist das Gegenstück zum Schreiben.

textLooksLikeWifi() prüft nur das Präfix WIFI: – daran erkennt die Hintergrundabfrage eine WLAN-Karte, ohne sie vollständig zu zerlegen.

Setup-Portal

SSID	C64uRemote-Setup
Passwort	c64ultimate
Adresse	192.168.4.1
Zeitlimit	5 min ohne Benutzung
Nachlauf nach dem Speichern	2,5 s, damit die Antwortseite noch ankommt

Ein DNSServer beantwortet alle Namen mit der eigenen Adresse, onNotFound leitet auf die Startseite um – zusammen ergibt das ein Captive Portal, das auf den meisten Geräten von selbst aufgeht.

Leer gelassene Passwortfelder lassen den bisherigen Wert stehen. Ist die SSID bereits gespeichert, wird deren Passwort übernommen.

NFC-Subsystem

Abfragestrategie

serviceRfid() kennt zwei Betriebsarten:

Auf den Kartenbildschirmen (NfcRead, NfcWrite, WifiCard) wird alle 250 ms mit cardPresent() gefragt – volles Zeitfenster des Lesers.

Auf dem Startbild läuft die Hintergrundabfrage im Abstand aus *Auto-NFC* (1,5 s / 0,7 s / 0,3 s) und benutzt cardPresentQuick().

Der Unterschied ist wichtig für die Bedienbarkeit: Liegt keine Karte auf, wartet der MFRC522 nach dem REQA-Kommando, bis sein interner Timer abläuft. PCD_Init() stellt dafür 0x03E8 = 1000 Schritte zu je 25 µs ein – **25 ms**, in denen die Hauptschleife steht. Eine Karte antwortet weit schneller (Frame Delay Time bei 106 kBit/s rund 86 µs), deshalb setzt cardPresentQuick() das Zeitfenster für die reine Probe auf 80 Schritte (2 ms) herunter und unmittelbar danach – noch vor der Kartenauswahl – wieder auf den Ausgangswert. Authentifizierung, Lesen und Schreiben laufen dadurch unverändert mit vollem Zeitfenster.

Eine liegen gebliebene Karte wird durch eine Sperre von 2,5 s (kRfidRepeatMs) nicht ständig erneut ausgeführt. Ausgenommen ist die Rückfrage einer PowerOff-Karte: die darf jederzeit durch erneutes Auflegen bestätigt werden.

Kartentypen

Familie	Zugriff
NTAG213/215/216, MIFARE Ultralight	4 Byte je Seite, NDEF-TLV ab Seite 4, kein Schlüssel
MIFARE Classic 1K/4K/Mini	16 Byte je Block, NDEF-TLV ab Block 4, Sektor-Trailer werden übersprungen

Bei Classic-Karten wird zuerst mit dem NDEF-Schlüssel D3F7D3F7D3F7 authentifiziert, ersatzweise mit dem Werksschlüssel FFFFFFFFFF. Der zuletzt erfolgreiche Schlüssel wird pro Karte gemerkt, sonst kostet jeder Block einen unnötigen Fehlversuch samt Neuauswahl.

Sektor-Trailer und MAD werden nie beschrieben. Eine Karte kann durch dieses Programm also nicht unbrauchbar werden; eine unformatierte Classic-Karte ist danach unter Umständen nur an diesem Gerät lesbar.

Nach einem fehlgeschlagenen Auth liegt die Karte im HALT-Zustand und antwortet nur noch auf WUPA. reselectCard() weckt sie deshalb gezielt mit PICC_WakeupA() und wählt sie neu aus – ein einfaches PICC_IsNewCardPresent() (REQA) würde sie nicht mehr finden.

Datenformat

Geschrieben wird immer ein einzelner NDEF-Textrecord, UTF-8, Sprachcode en:

```
TLV      : 03 <len> ... FE
Record   : D1 01 <plen> 54 | 02 'e' 'n' | <text>
           D1 = MB|ME|SR|TNF=1 (Well Known), 54 = 'T'
```

Der Puffer wird auf ein Vielfaches von 16 aufgefüllt, damit immer ganze Blöcke geschrieben werden. Nach dem Schreiben liest writeCardText() die Karte zur Kontrolle wieder ein und vergleicht den Text.

Beim Lesen wird zusätzlich das alte Rohformat mit der Kennung C64UPATH erkannt, damit bereits beschriebene Karten weiter funktionieren.

Der Parser ist absichtlich tolerant: Manche Schreiber tragen eine falsche Payload-Länge ein – der TeensyROM schreibt dort konstant 0x10, obwohl die TLV-Länge stimmt. Beim letzten Record (ME-Flag) hat deshalb die TLV-Länge Vorrang.

Kommandokarten

```
CMD:RESET
CMD:REBOOT
CMD:MENU
CMD:POWEROFF=0      sofort ausschalten
CMD:POWEROFF=8      nachfragen, 8 s Zeit für die Bestätigung
CMD:POWEROFF        nachfragen mit der Geräteeinstellung "NFC-Cmd PowOff"
CMD:CPU=10          CPU auf 10 MHz stellen
CMD:JOY             Joystickports umschalten (Normal <-> Swapped)
CMD:JOY=SWAPPED     Ports fest setzen; auch NORMAL, WASD1, WASD2
```

parseCardCommand() ist unempfindlich gegen Groß-/Kleinschreibung und Leerzeichen. Ohne Argument gilt bei POWEROFF die Geräteeinstellung *NFC-Cmd PowOff* (3/5/8/15 s), 0 bedeutet „ohne Nachfrage“.

Die Rückfrage läuft über app.cardPowerOffPending samt UID und Ablaufzeit. Bestätigt wird durch erneutes Auflegen **derselben** Karte oder durch einen Tastendruck (activateCurrent() fängt das vor allem anderen ab).

Was der Stick nicht kann

processCard() erkennt Pfadkarten, führt sie aber nicht aus:

```
app.rfidHint = "Programmkarte - dafuer fehlt die SD";
setModal("BRAUCHT SD-KARTE", ...);
```

Für einen Programmstart müsste die Datei von einer lokalen microSD gelesen und per Streaming-Upload an den C64 geschickt werden. Beides gibt es nur in der Core- und der Dial-Fassung.

Softwarearchitektur

Zustandsmodell

ScreenMode steuert Darstellung und Eingabe:

Zustand	Inhalt
Home	Logo / Effekte, Hintergrundabfrage des Lesers
Menu	Hauptmenü
CpuMenu	Taktstufen
Status	Verbindungsübersicht
DisplaySettings	Setup-Liste
NfcMenu	Untermenü NFC / RFID
NfcRead	Karte auflegen → Inhalt ausführen
CmdPick	Befehl für eine Karte auswählen
NfcWrite	Karte auflegen → Text schreiben
WifiMenu	Untermenü WLAN
WifiCard	Karte auflegen → Zugangsdaten lesen
WifiSaved	gespeicherte Netze, wahlweise im Löschmodus
WifiPortal	Setup-Accesspoint läuft

Navigation

Damit ein neuer Bildschirm nicht jedes Mal drei weitere switch-Zweige nach sich zieht, laufen alle Eingaben über drei Funktionen:

```
void moveSelection(int delta);      // blättern
void activateCurrent(uint32_t);    // auswählen / ausführen
void goBack(uint32_t);            // eine Ebene zurück
```

listSelection() liefert dazu Zeiger und Länge der Liste des aktuellen Bildschirms. Tasten und MiniJoyC rufen nur noch diese drei Funktionen auf; die Sonderfälle des Startbilds (Doppelklick = Reboot, Joystick hoch = Reset) stehen direkt in der Tastenauswertung.

activateCurrent() fängt vor allem anderen eine offene PowerOff-Rückfrage ab.

Rendering

Gezeichnet wird immer in ein M5Canvas und danach in einem Stück auf das Display geschoben – dadurch flimmerfrei. Die Bildrate liegt bei rund 30 fps (kFrameMs = 33).

Auf dem Startbild wird nur neu gezeichnet, wenn sich etwas geändert hat (app.home.frameDirty) oder ein Effekt läuft. Die Effekte lesen aus plainLogoPixels, dem vorgerenderten Logo; boxedLogoPixels enthält dieselbe Grafik mit freigestelltem Innenbereich.

ReST-Anbindung an den C64 Ultimate

Basis ist <http://<host>>, Zeitlimit 2,5 s. Ist ein Passwort hinterlegt, geht es als Kopfzeile X-Password mit.

Zweck	Methode und Pfad
Erreichbarkeit / Version	GET /v1/version
Reset	PUT /v1/machine:reset
Reboot	PUT /v1/machine:reboot
Ausschalten	PUT /v1/machine:poweroff
Ultimate-Menü	PUT /v1/machine:menu_button
Konfigurationsbaum	GET /v1/configs
CPU-Wert lesen	GET /v1/configs/<Kategorie>/<Eintrag>
CPU-Wert setzen	PUT /v1/configs/<Kategorie>/<Eintrag>?value=<Wert>
Joystickports lesen/setzen	dieselben /v1/configs-Pfade

Den Pfad zur CPU-Einstellung sucht resolveCpuPath() einmalig im Konfigurationsbaum – die Ultimate-Firmware benennt ihn je nach Version unterschiedlich. Gelingt das nicht, greift eine fest eingebaute Ersatzliste der üblichen Taktstufen.

Fuer die Joystickports gibt es keinen machine:-Befehl. Sie sind ebenfalls ein Konfigurationseintrag, im Test *Joystick Swapper* in *U64 Specific Settings* mit den Werten Normal, Swapped, WASD Port 2, WASD Port 1. resolveJoyPath() sucht ihn nach demselben Muster (zuerst *U64 Specific Settings*, sonst alle Kategorien nach einem Eintragsnamen mit „Joystick“). joyTokenFromValue() und joyValueFromToken() rechnen zwischen Geraetewert und Kartenkuerzel um (WASD Port 1 <-> WASD1). Achtung beim Lesen des Quelltexts: alles andere mit joy im Namen gehoert zum MiniJoyC am HAT-Port.

refreshConnectionStatus() fragt höchstens alle 15 s nach, zuerst ohne und dann mit Passwort. Daraus ergeben sich die vier Zustände der Status-LED.

Bedienlogik

Tasten

Taste	Startbild	Listen
A	Reset, zweimal innerhalb 300 ms = Reboot	goBack()
B	Ultimate-Menü	moveSelection(+1)
Power (kurz)	Menü öffnen	activateCurrent()

Der Einzelklick auf A wird bewusst verzögert ausgeführt: Erst wenn nach 300 ms kein zweiter Druck kommt, läuft der Reset. Anders ließen sich Reset und Reboot nicht auf dieselbe Taste legen.

Blockierende Aufrufe

Zwei Stellen halten die Hauptschleife bewusst an:

- **HTTP-Anfragen** bis zu 2,5 s
- **Warten auf die WLAN-Verbindung** nach einer WLAN-Karte bis zu 8 s (kWifiCardConnectMs), mit vorher gezeichneter Meldung *VERBINDE...*

In beiden Fällen ist eine kurze Blockade der ehrlichere Weg: Der Nutzer soll das Ergebnis sehen, nicht ein Menü, das sich weiterbedienen lässt, während im Hintergrund noch nichts feststeht.

PowerOff-Absicherung

Ausschalten ist an drei Stellen abgesichert:

1. Menüpunkt und Joystick-Runter fragen immer nach - zweiter Druck innerhalb von 2 s (kPowerOffConfirmMs)
2. Befehlskarten mit Abfrage: erneutes Auflegen oder Tastendruck innerhalb von 3-15 s
3. CMD:POWEROFF=0 schaltet ohne Nachfrage - wer das anlegt, will es so

Persistenz

Was	Wo
Anzeige- und Bedieneinstellungen	NVS c64uremote
WLAN-Profil, Host, Host-Passwort	NVS c64unet
Erkanntes Board	NVS, von M5GFX selbst verwaltet

Factory Reset im Setup betrifft nur c64uremote. Die Netzdaten löscht man über *WLAN* → *Alle löschen*, alles zusammen mit `pio run -t erase`.

Projektstruktur

M5StickC_Plus2_C64uRemote/	
├─ platformio.ini	Board, Bibliotheken, Partition, Upload
├─ README.md	
├─ LICENSE	MIT - Karl Prosser, Martin Oswald
├─ .vscode/	empfohlene Erweiterungen, Editor-Einstellungen
├─ docs-src/	Markdown-Quellen der Handbücher
├─ doc/	fertige Handbücher (PDF)
├─ src/	deutsche Fassung
│ └─ main.cpp	gesamte Firmware
│ └─ build_env.h	Zugangsdaten (nicht versionieren)
│ └─ build_env.h.example	Vorlage
│ └─ 1MHz_logo_rgb565.h	Logo als RGB565-Feld
└─ src-en/	
└─ main.cpp	englische Fassung, Code identisch

Abhängigkeiten: M5Unified, M5GFX, ArduinoJson und der I²C-Treiber `kkloesener/MFRC522_I2C`. Für den MiniJoyC gibt es bewusst keine Bibliothek - die vier benötigten Zugriffe stehen direkt im Quelltext.

Fehlerdiagnose

Bildschirm bleibt dunkel. `platformio.ini` gegen das Kapitel *Board-Einstellungen* prüfen. Im seriellen Monitor muss `[Autodetect] M5StickCPlus2` stehen. Hilft das nicht: `pio run -t erase`, dann neu flashen.

Meldung WENIG SPEICHER. Die Logo-Caches passten nicht in den Speicher. Prüfen, ob `DBOARD_HAS_PSRAM` gesetzt ist; im Monitor steht sonst `logo cache alloc failed` samt freiem Heap.

Unter NFC / RFID steht überall kein NFC. Der Leser wurde beim Start nicht gefunden. Grove-Stecker prüfen; im Monitor steht `RFID2 nicht gefunden`. Der Leser wird nur einmal beim Start gesucht – nach dem Anstecken also neu starten.

Karten werden gelesen, aber nicht geschrieben. Bei MIFARE Classic sind die Datenblöcke unter Umständen mit einem fremden Schlüssel gesichert. Das Programm probiert nur den NDEF- und den Werksschlüssel; ein Wörterbuchangriff steckt bewusst nur in der Core-Fassung.

Das Setup-Portal geht nicht auf. Manche Telefone verwerfen ein WLAN ohne Internetzugang selbsttätig. Dann in den WLAN-Einstellungen „trotzdem verbunden bleiben“ wählen und `192.168.4.1` von Hand im Browser eingeben.

Quellen

- Originalprojekt: Karl Prosser (@klumsy), <https://github.com/ReadyOS-C64/C64uRemote>
- Ultimate-Firmware und ReST-API: <https://1541u-documentation.readthedocs.io>
- M5Unified / M5GFX: <https://github.com/m5stack/M5Unified>
- MFRC522-I²C-Treiber: https://github.com/kkloesener/MFRC522_I2C
- Diese Fassung: Martin Oswald (@mad), <https://1MHz.de>

Lizenz

C64uRemote steht unter der **MIT-Lizenz**. Der vollständige Lizenztext liegt als Datei `LICENSE` im Projektstamm.

Ursprung ist das Projekt **C64uRemote von Karl Prosser (@klumsy)**, <https://github.com/ReadyOS-C64/C64uRemote>, das er unter der MIT-Lizenz veröffentlicht hat. Diese Fassung ist eine daraus abgeleitete Erweiterung und steht unter denselben Bedingungen:

- Copyright (c) 2026 Karl Prosser – Originalprojekt
- Copyright (c) 2026 Martin Oswald (@mad, <https://1MHz.de>) – Portierung und Erweiterungen

Die MIT-Lizenz erlaubt es, die Software zu benutzen, zu verändern und weiterzugeben, auch kommerziell. Einzige Bedingung: **Copyright-Vermerk und Lizenztext müssen erhalten bleiben** und jeder Kopie beiliegen. Eine Gewährleistung oder Haftung ist ausgeschlossen.

Die eingebundenen Bibliotheken haben ihre eigenen Lizenzen: M5Unified und M5GFX (MIT, © M5Stack), ArduinoJson (MIT, © Benoit Blanchon) sowie MFRC522_I2C (https://github.com/kkloesener/MFRC522_I2C). Sie werden beim Bauen von PlatformIO geladen.

Entstehung

Portierung, Erweiterungen und Handbücher sind mit Unterstützung von Claude (Anthropic) entstanden. Konzept, Idee, Hardware-Entscheidungen und sämtliche Tests auf den echten Geräten: Martin Oswald (@mad).